Operating System (25CS2104E)

Readers–Writers Concurrent Database Simulator

Team Members

SAI ANANYA ROY 2520030471

D. SHREENITHA 2520030504

Under the Esteemed Guidance of

Dr.Hema

Contents

- Abstract
- Introduction
- Problem Statement and Objectives
- Flow Chart
- Technologies Used
- Woking Proces
- Implementation and Output
- Conclusion & Future Improvements

Abstract

The Readers–Writers Database Simulator is developed using C to demonstrate how multiple readers and a writer access shared student data. The writer can enter or update a student's roll number, subject, and marks. Two readers can view the same records and detect changes automatically. Shared memory and synchronization are used to keep the data consistent. The project demonstrates concurrency and safe access to shared resources.

Introduction

This project simulates a student marks database where information can be updated and viewed through separate terminals. The writer manages student records, while Reader 1 and Reader 2 display the data. Whenever the writer updates marks, both readers can see the latest information. The project helps us understand threads, shared memory, and synchronization in operating systems.

Problem Statement and Objectives

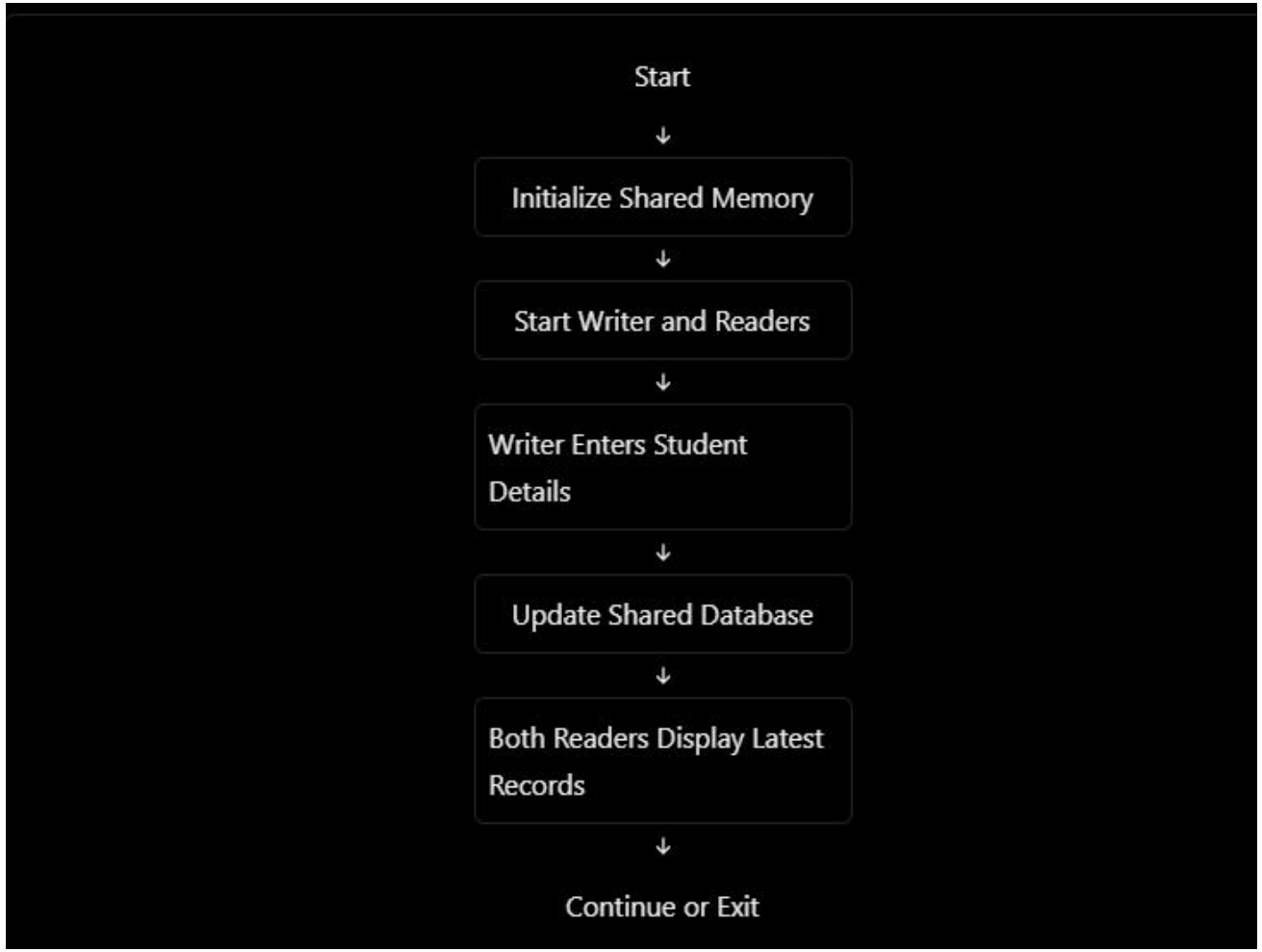
Problem Statement

When multiple users access the same data, updates must be managed carefully to avoid incorrect or inconsistent information.

Objectives

- Allow the writer to add and update student marks.
- Store roll number, subject, and marks.
- Allow two readers to view the same records.
- Reflect updates in both reader terminals.
- Maintain data consistency using synchronization.

Flow Chart


Start
Initialize Shared Memory
Start Writer and Readers
Writer Enters Student Details
Update Shared Database
Both Readers Display Latest Records
Continue or Exit

Technologies Used

- C Programming: Implements the writer and reader programs.
- Ubuntu Linux: Environment used to run the project.
- GCC Compiler: Compiles the C source files.
- POSIX Shared Memory: Shares student records between separate processes.
- POSIX Synchronization: Coordinates access to shared data and prevents conflicting updates.
- Terminal: Allows the writer to enter data and readers to display it.

Working Process

- Start the writer in Terminal 1.
- Start Reader 1 in Terminal 2 and Reader 2 in Terminal 3.
- Enter the roll number, subject, and marks in the writer terminal.
- The writer adds or updates the shared student record.
- Both readers detect the change and display the latest records.
- The writer can continue updating marks while the readers monitor the database.

Implementation and Output

Implementation

- Separate C files are used for the writer and reader programs.
- Shared memory stores student records.
- Synchronization protects records during updates.
- Two reader processes monitor the same database.

```
shreenitha@shreenitha2203:~/student_marks$ nano shared.h
shreenitha@shreenitha2203:~/student_marks$ nano writer.c
shreenitha@shreenitha2203:~/student_marks$ nano reader.c
shreenitha@shreenitha2203:~/student_marks$ nano writer.c
shreenitha@shreenitha2203:~/student_marks$ gcc writer.c -o writer -pthread -
lrt
gcc reader.c -o reader -pthread -lrt
shreenitha@shreenitha2203:~/student_marks$ ./writer

===== STUDENT MARKS WRITER =====
Enter roll number 0 to stop.

Enter roll number: 103
Enter subject: english
Enter marks (0-100): 92
New student subject record added.
Shared database updated. Readers will see the change.

Enter roll number: 103
Enter subject: english
Current marks: 92
Do you want to update marks? (y/n): n
Marks remain unchanged.

Enter roll number:
```

```
shreenitha@shreenitha2203:~$ cd ~/student_marks
shreenitha@shreenitha2203:~/student_marks$ ./reader

===== STUDENT MARKS READER =====
Waiting for student records and updates...

Roll No.   Subject              Old Marks    New Marks
-----------------------------------------------------
103        english              92           92
Waiting for next update...
```

Conclusion and Future Improvements

Conclusion

- Demonstrates the Readers–Writers problem using student records.
- Allows two readers to view shared data.
- Shows how updates can be reflected across separate terminals.
- Explains the importance of synchronization and data consistency.

Future Improvements

- Add more student details and subjects.
- Add student search by roll number.
- Save records permanently using a file or database.
- Develop a graphical user interface for easier use.

(DEEMED TO BE UNIVERSITY)

Thank you